ODISSEY
Tech, S.A. de C.V.

# Manual de despliegue de proyecto

**Proyecto:** Sistema de Preventas
**Fecha:** 16 de abril del 2026
**Autor:** Odyssey Team

**Herramientas principales:**

**Supabase**
**Railway**

# Manual de despliegue en Supabase:

Requisitos:
Cuenta activa en supabase

Supabase proporciona la base de datos en Postgres y las credenciales necesarias para conectar la aplicación

## 1) Iniciar un proyecto:

1. Ingresar a supabase en: https://supabase.com
2. Iniciar sesión con una cuenta activa
3. Seleccionar “New project”
4. Completar los campos:
   Organization: Nombre de la organización
   Name: Nombre del proyecto
   Database Password: genera una contraseña y la guarda
   Region: elegir región cercana a los usuarios
5. Seleccionar “Create new project”

## 2) Obtener las credenciales de conexión:

En el dashboard de Supabase ve a → Project Settings → API:

- URLs del proyecto:

SUPABASE_URL: Project URL (ej. https://xxxx.supabase.co)

Para obtenerlo ve a → Project settings → Data API → API URL

DATABASE_URL:
postgresql://postgres:[PASSWORD]@db.xxxx.supabase.co:5432/postgres

Para obtenerlo ve a → Menu principal → Connect → Direct → Transaction pooler

- LLave para el cliente:

SUPABASE_KEY: anon public key (para el cliente)

Para obtenerlo ve a → Project settings → API Keys → Publishable Key

## 3) Crear el esquema en la base de datos

En Supabase ve a SQL Editor y ejecuta los archivos que están en /supabase_sql del proyecto en el siguiente orden:

Prioridad 1 = Mayor prioridad
Prioridad 4 = Menor prioridad

Prioridad 1: Tablas sin dependencias (independientes)

1. rol
2. privilegio
3. producto
4. campania
5. sucursal
6. cuenta
7. configuraciones
8. session

Prioridad 2: Tablas con una dependencia

9. usuario → depende de: rol
10. rol_privilegio → depende de: rol, privilegio
11. producto_campania → depende de: producto, campania
12. sucursal_cuenta → depende de: sucursal, cuenta

Prioridad 3: Tablas con dos dependencias

13. orden → depende de: usuario, campania, sucursal
14. usuario_cuenta → depende de: usuario, cuenta
15. logs → depende de: usuario

Prioridad 4: Tablas que dependen de tablas con prioridad 3

16. detalle_orden → depende de: orden, producto
17. calificacion → depende de: producto, usuario

## 4) Configurar Row Level Security (RLS) *Importante para tener acceso a los datos de las tablas*

Supabase activa RLS por defecto. Tienes dos opciones:

Opción A — Desactivar RLS (más simple, solo recomendado en desarrollo):
ALTER TABLE usuario DISABLE ROW LEVEL SECURITY;
ALTER TABLE producto DISABLE ROW LEVEL SECURITY

Repetir esto para cada tabla.

Opción B — Usar la service_role key (recomendado para producción desde backend):
En Settings → API, copia la service_role key (secreta, importante nunca exponerla al frontend).
Úsala como SUPABASE_KEY en el .env del servidor — esta key bypasea RLS.

En el proyecto usamos Supabase sólo desde el backend, así que la service_role key es la opción correcta para producción.

## 5) Instalar dependencias para poder arrancar

Instalar dependencias: “npm install” o “npm i”
Iniciar: “npm start” o “node app.js”

# Manual de Despliegue en Railway

## 1. Creación de cuenta y vinculación con GitHub

1. Acceder a Railway: https://railway.app
2. Crear una cuenta o iniciar sesión.
3. Seleccionar la opción **“Login with GitHub”**.
4. Autorizar el acceso a la cuenta de GitHub donde se encuentra el repositorio del proyecto.

## 2. Creación de un nuevo proyecto

1. Dentro del dashboard de Railway, hacer clic en **“New Project”**.
2. Seleccionar la opción **“Deploy from GitHub Repo”**.
3. Elegir el repositorio correspondiente al proyecto.
4. Esperar a que Railway realice el build automático inicial.

## 3. Configuración de variables de entorno

Aunque Railway puede detectar automáticamente el archivo `.env`, es necesario verificar manualmente las variables en la sección **“Variables”** del proyecto.

Configurar o validar las siguientes variables:

MAIL_USER=odyseyteam5@gmail.com
MAIL_PASS=tu_password_de_aplicacion
SESSION_SECRET=string_largo_random
NODE_ENV=production

## 4. Obtención de MAIL_PASS (contraseña de aplicación)

Para el envío de correos, no se debe utilizar la contraseña normal del correo, sino una **contraseña de aplicación**.

Pasos para generarla:

1. Acceder a la cuenta de Google.
2. Activar la verificación en dos pasos.
3. Buscar la opción **"Contraseñas de aplicación"**.
4. Generar una nueva contraseña para la aplicación de correo.
5. Google proporcionará un código de 16 caracteres.
6. Utilizar ese código como valor de `MAIL_PASS`.

## 5. Configuración importante

- `NODE_ENV` debe estar en **production** (no "development").
- `SESSION_SECRET` debe ser una cadena larga y aleatoria (mínimo 32 caracteres).
- Verificar que las credenciales de correo funcionen correctamente.

## 6. Configurar script de inicio en package.json

Abre tu archivo `package.json` y asegúrate de tener esto en la sección de scripts:

```
▷Debug
"scripts": {
  "start": "node app.js",
```

## 7. Configurar el puerto dinámico en tu servidor

En tu archivo principal (`app.js`), verificar el puerto:

```
const PORT = process.env.PORT || 3000;

app.listen(PORT, () => {
  console.log(`Servidor corriendo en el puerto ${PORT}`);
});
```

## 8. Generación de dominio

1. Dentro del proyecto en Railway, acceder a la sección de configuración.
2. Ir a **Networking**.
3. Seleccionar la opción **“Generate Domain”**.
4. Railway asignará una URL pública para acceder a la aplicación.